Interrupt (Theory)

Definition of Interrupt

An **interrupt** is a signal sent to the **CPU** that temporarily stops the execution of the current program so that the processor can handle an urgent task.

After handling the interrupt, the CPU **returns to the original program** and continues its execution.

Interrupts help the computer respond quickly to external events.

Interrupt Service Routine (ISR)

An **Interrupt Service Routine (ISR)** is a special program or function that is executed by the CPU when an interrupt occurs.

The ISR performs the task required to handle the interrupt.

Steps of Interrupt Handling

1. An interrupt signal is generated.
2. The CPU stops the current program temporarily.
3. The CPU saves the current program state.
4. The CPU executes the **Interrupt Service Routine (ISR)**.
5. After completing the task, the CPU returns to the previous program.

Types of Interrupts

1. Hardware Interrupt

Hardware interrupts are generated by **external devices** connected to the computer system.

Examples:

- Keyboard input
- Mouse click
- Printer request
- Disk operations

These interrupts inform the CPU that a device needs attention.

---

2. Software Interrupt

Software interrupts are generated by **program instructions**.

They are used by programs to request services from the operating system.

Example:

- System calls

---

3. Maskable Interrupt

Maskable interrupts are the interrupts that **can be ignored or disabled by the CPU** when necessary.

They are usually used for **normal device operations**.

---

4. Non-Maskable Interrupt (NMI)

Non-maskable interrupts are **very important interrupts that cannot be ignored by the CPU**.

They are used for **critical system problems**, such as hardware failures or serious errors.

Advantages of Interrupts

- Improves CPU efficiency
- Reduces idle waiting time
- Allows faster response to external devices
- Supports multitasking

Disadvantages of Interrupts

- Makes system design more complex
- Causes context switching overhead
- Too many interrupts can slow down the CPU

Memory Addresses

Memory Address

A **memory address** is the unique number that identifies a specific location in computer memory where data or instructions are stored.

Each memory cell has its **own address** so the CPU can easily find and access the required data.

Example:

Memory Address	Data
1000	25
1001	40
1002	75

Memory Location

A **memory location** is a small storage unit in memory that stores data.

Each location is identified by a **unique memory address**.

Address Bus

The **address bus** is a set of electrical lines used by the CPU to send memory addresses to memory.

Characteristics:

- It transfers **address information only**
- It is **unidirectional** (from CPU to memory)
- The number of address lines determines the memory size

Memory Address Formula (Important for Numericals)

The total number of memory locations that a system can access is calculated using:

Number of Memory Locations=2n\text{Number of Memory Locations} = 2^nNumber of Memory Locations=2n

Where:

n = number of address lines

Example

If a computer system has **16 address lines**:

216=655362^{16} = 65536216=65536

So the system can access **65536 memory locations**.

Addressing Modes

Addressing modes define **how the CPU finds the operand (data)** in memory.

1. Immediate Addressing

In this mode, the data is **directly written in the instruction**.

Example

```
MOV A, 5
```

The value **5** is directly used.

---

2. Direct Addressing

In direct addressing, the **memory address of the data is given in the instruction**.

Example

```
MOV A, [1000]
```

The CPU retrieves the data stored at memory address **1000**.

---

3. Indirect Addressing

In indirect addressing, the **address of the data is stored in a register**.

Example

```
MOV A, [R1]
```

Here the register **R1 contains the memory address**.

---

4. Register Addressing

In this mode, the operand is stored **directly inside a CPU register**.

Example

```
MOV A, B
```

Data is copied from register **B to register A**.